

M9 — Modernizing Data Warehouses with BigQuery and Lakehouse: Student Notes

Notes for the Day 2 (Tue Oct 6) late-morning module, 11:35–12:00 — why your SQL Server data warehouse pain points disappear in BigQuery, and your first look at partitioning, clustering, and Iceberg table internals.

Key takeaways

First — the 'traditional data warehouse challenges' slide is about you. The deck lists a fictional retailer's architects' problems. Read them as a checklist of your own SQL Server life:

- ☐ Extensive time spent on capacity planning for peak events (your nightly extract window, month-end, open enrollment)
- ☐ Difficulty handling unexpected load spikes — the box is sized for average, or sized for peak and idle the rest of the day
- ☐ Manual provisioning: hardware procurement, software installation, patching weekends
- ☐ Rebalancing data manually as it grows (new drives, filegroup shuffles, archiving old partitions by hand)
- ☐ Operational overhead that is expensive and slows down how fast the analytics teams get answers

Every one of those is an infrastructure problem, not a data problem. BigQuery's answer is to delete the infrastructure layer:

- **BigQuery is fully managed and serverless.** No hardware, no patching, no capacity planning. You load data and run queries; Google allocates the compute and scales it up or down per query.
- **Storage and compute are separate.** This is the architectural fact everything else follows from. Storage scales automatically as data grows; compute scales when a query needs more power — and you pay for each independently. In your SQL Server world, growing the data and growing the horsepower are the same painful purchase.
- **Slots = the unit of compute.** A slot is a small bundle of CPU, RAM, and network. When you run a query, BigQuery's engine (Dremel) assigns *thousands* of slots to it, each chewing a small piece of the data simultaneously. That is how a full scan of a billion-row table returns in seconds — not a faster server, a wider one.
- **Shuffle = the re-sort between steps.** For GROUP BY and JOIN, intermediate results get redistributed from the slots that produced them to the slots that will aggregate or join them. BigQuery does this over an in-memory shuffle tier and a petabit network. You will see "bytes shuffled" in a query's execution details — it is the cost of moving data between workers, and large shuffles are where slow queries hide.
- **Partitioning prunes whole date ranges.** Partition a table (usually by a date column) and a query filtered on that date reads only the matching partitions — fewer bytes scanned, lower cost, faster results.

- **Clustering prunes within a partition.** Clustering sorts the data inside each partition by up to four columns you choose; queries filtered on those columns skip the storage blocks that can't contain matching rows.
- **Iceberg formalizes all of this in open metadata.** For Iceberg tables over files in Cloud Storage, the Iceberg metadata tracks which files belong to which partition, plus min/max statistics per file — so filtering works on open files in your bucket the same way it works on native BigQuery storage. This is called predicate pushdown.
- **This module is a preview.** Partitioning, clustering, and Iceberg internals get deep, hands-on treatment in the 3-day *Data Warehousing with BigQuery* course being scheduled for your team. Today you need the vocabulary and the decision rules, not every knob.

BigQuery fundamentals, in plain terms

The mental model to keep: **BigQuery is not a bigger SQL Server. It is a query engine and a storage system that happen to share a SQL front door.**

- The storage side is replicated, distributed, and separate from compute (99.99% durability). Bulk loading is free; streaming ingest exists for when you need it.
- The compute side is a high-availability cluster that borrows thousands of slots per query, then gives them back. SQL:2011 compliant (GoogleSQL), reachable from the console, CLI, REST API, and client libraries.
- Because the two sides are separate, multisource querying is natural: the same engine can read native BigQuery storage, files in Cloud Storage, and federated sources — which is exactly the lakehouse pattern you worked with this morning (DIN §30 external tables) and again in this afternoon's lab.

Partitioning and clustering — the mini decision guide

Think of partitioning as the drawers of a filing cabinet (one drawer per day, month, or year) and clustering as sorting the files *inside* each drawer.

Question	Default answer
Should this big table be partitioned?	Almost always yes, on the date column your reports filter on (event date, load date). Partitioning is what makes bytes-scanned predictable before you hit run.
Which granularity?	Daily is the default. Go monthly/yearly if daily partitions would be tiny (rule of thumb: aim for at least ~1–10 GB per partition — many small partitions slow metadata, not queries).
Should it also be clustered?	Yes, on the columns you filter and

Question	Default answer
	join on most — up to four, in priority order (e.g., county_id, customer_id). High-cardinality filter columns benefit most.
When is clustering enough without partitioning?	When the table is modest, partitions would be small, or your filters span many different columns rather than one date.

Two things that will feel different from SQL Server:

- You declare partitioning and clustering at CREATE TABLE time; there is no ALTER that retroactively repartitions an existing table (you rebuild it with a CREATE TABLE ... AS SELECT).
- You never rebuild or reorganize these. BigQuery re-clusters in the background automatically as data arrives. The index-maintenance half of your M&O checklist simply does not exist here.

Iceberg partitioning and predicate pushdown

The same pruning story works on open files in your bucket. With Iceberg tables, the partition scheme and per-file min/max statistics live in Iceberg metadata (written by whichever engine produced the table — Spark, or BigQuery itself). When your query filters WHERE transaction_date = '2025-08-15', BigQuery reads that metadata, identifies the exact files that can contain that date, and never opens the rest. Filter on a clustered/sorted column and the per-file statistics let it skip files whose min/max range excludes your value. Same discipline as partition elimination and index seeks in SQL Server — but the "indexes" are open metadata sitting next to your Parquet files, readable by any engine that speaks Iceberg.

Your SQL Server world → GCP world (this module)

In your current stack	In this module's GCP world
Sizing the server for the nightly window / peak season	Serverless slots — BigQuery allocates thousands per query, then releases them
Storage and compute bought as one box	Storage and compute scale (and bill) independently
Patching, hardware refresh, drive rebalancing (M&O)	Fully managed — none of these exist
Table partitioning (partition functions/schemes, sliding windows)	BigQuery partitioned tables — pruning is automatic on a filter
Clustered indexes and index tuning	Clustering columns — sort order inside storage blocks
Index rebuilds / reorgs in the	Automatic background reclustering —

In your current stack	In this module's GCP world
maintenance plan	no job to write
Staging database for file drops	Lakehouse tables over files in Cloud Storage (this afternoon's lab)
Resource Governor	Slot reservations / capacity pricing (a billing choice, not a config file)

Dig deeper

- [Introduction to partitioned tables](#)
- [Introduction to clustered tables](#)
- [Understanding BigQuery slots](#)
- [Apache Iceberg managed tables in BigQuery](#)

Part of the Iowa DOM DoIT Data Engineering student pack — see ../00_schedule.md.